

Python para Economistas

Universidad Nacional de La Plata

14 de octubre de 2022

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares
- 4 Widgets en Jupyter

En la clase pasada vimos:

- ① NumPy
 - Creación de arrays y operaciones.
 - Números aleatorios.
- ② Pandas: manipulación de datos tabulares.
- ③ Matplotlib: generación de gráficos.

Objetivos de la clase

En la clase de esta semana cubriremos los siguientes temas:

- **Profundización en pandas:** vamos a avanzar sobre lo visto en la clase 3 y cubrir algunas operaciones más complejas.
- **Visualización con seaborn:** vamos a estudiar este paquete derivado de `matplotlib`, que tiene una interfaz más amigable para trabajar con `pandas`.
- **Expresiones regulares:** es una herramienta muy poderosa para el procesamiento de texto.
- ***Widgets*:** es una herramienta muy útil para dotar a nuestros notebooks de Jupyter de reactividad.

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares
- 4 Widgets en Jupyter

Pandas (continuación de la clase 3)

En la clase 3 vimos:

- Cómo crear un **DataFrame** desde cero.
- Cómo cargar datos de un archivo tabular en un **DataFrame**.
- Cómo filtrar filas y/o columnas.
- Combinar **DataFrame**'s en base a los valores de una o más columnas.
- Agrupar de acuerdo a una o varias columnas y generar agregaciones a nivel de los grupos generados (por ejemplo: cuenta, suma, promedio)

En esta parte de la clase vamos a cerrar el tema de Pandas, viendo algunos conceptos un poco más avanzados.

Otras operaciones con DataFrame's agrupados

En la clase 3, vimos cómo computar estadísticas agregadas a nivel de grupos, utilizando el método `groupby` seguido de una operación de agregación en el objeto resultante.

Otras cosas que podemos querer hacer a nivel de grupos son:

- **Transformaciones.** Por ejemplo:
 - estandarizar una variable de acuerdo a la distribución que tiene condicionada al grupo.
 - rellenar datos faltantes con, por ejemplo, la media del grupo.
- **Filtrado:** descartar observaciones que pertenecen a grupos con determinadas características. Por ejemplo:
 - descartar filas pertenecientes a grupos con pocas observaciones.
 - descartar filas donde un estadístico a nivel de grupo (como el promedio) cumple cierta condición.

Transformaciones sobre grupos

Ejemplo. Estandarizar columnas usando la media y la desviación estándar de cada grupo.

```
1 df.groupby().transform(lambda x: (x - x.mean())/x.std())
```

Ejemplo desarrollado en el notebook.

Filtrado por grupos

Ejemplo. Eliminar filas pertenecientes a grupos con menos de 5 observaciones:

```
1 df.groupby().filter(lambda x: len(x) < 5)
```

Ejemplo desarrollado en el notebook.

Pandas: reformateando DataFrame's

La forma canónica de estructurar un **DataFrame** es poner las *observaciones* en las filas, y las distintas *variables* observadas en las columnas.

Sin embargo, qué se considera una observación exactamente no está bien definido, y la estructura que le demos a nuestro **DataFrame** va a quedar determinada en gran medida por lo que queramos hacer después.

Pandas: reformateando DataFrame's

Pasar de un DataFrame 'ancho' a uno 'largo':

df3

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150



df3.melt(id_vars=['first', 'last'])

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130
3	Mary	Bo	weight	150

Pregunta 1 (Cuestionario)

Supongamos que tenemos el DataFrame `df`:

	Nombre	Apellido	Edad	Dirección	Ciudad
0	Mabel	García	40	San Martín 900	La Plata
1	Juan	Pérez	50	Belgrano 1770	Rosario
2	Héctor	Rossi	60	Sarmiento 1888	Santa Rosa

¿Qué dimensiones (`shape`) tiene el resultado de hacer `df.melt(id_vars=['Nombre', 'Apellido'])`? Expresarlo como `filas × columnas`.

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares
- 4 Widgets en Jupyter

Visualizaciones (continuación)

Vamos a estudiar el paquete **seaborn** para gráficos.

seaborn está basado en **matplotlib** y ofrece una interfaz más amigable para trabajar con objetos como **DataFrame**'s de **pandas**. Esto permite realizar gráficos relativamente complejos con un código relativamente simple.

Lo podemos instalar de esta manera:

```
conda install -c anaconda seaborn
```

Una vez instalado, lo importamos así¹:

```
1 import seaborn as sns
```

¹Como siempre, usar el alias al importar no es necesario pero es una práctica habitual.

Un gráfico sencillo con seaborn

```
1 sns.set_theme()
```

Pregunta 2 (cuestionario)

Dado el `DataFrame` de propinas (supongamos que se llama `propinas_df`) que vimos en el ejemplo:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

¿Qué línea de código en **seaborn** escribirías para generar un boxplot que tenga en el eje horizontal el tamaño de la mesa y en el eje vertical el valor de la propina?

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares**
- 4 Widgets en Jupyter

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).
- Compiladores o intérpretes de lenguajes de programación (por ejemplo, el intérprete de Python!)

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).
- Compiladores o intérpretes de lenguajes de programación (por ejemplo, el intérprete de Python!)
- Validación de entrada de usuario (por ejemplo, verificar que un string es una dirección de e-mail válida).

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).
- Compiladores o intérpretes de lenguajes de programación (por ejemplo, el intérprete de Python!)
- Validación de entrada de usuario (por ejemplo, verificar que un string es una dirección de e-mail válida).
- Minado de datos.

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).
- Compiladores o intérpretes de lenguajes de programación (por ejemplo, el intérprete de Python!)
- Validación de entrada de usuario (por ejemplo, verificar que un string es una dirección de e-mail válida).
- Minado de datos.

Expresiones regulares

En Python, el módulo `re` se utiliza para trabajar con expresiones regulares. No hace falta instalarlo, pues viene incluido en la librería estándar.

Para trabajar con él, ejecutamos

```
1 import re
```

¿Cómo defino una expresión regular?

Empecemos con una expresión regular sencilla:

```
1 >>> import re
2 >>> frase = "    Dnde    va Vicente? Donde va la gente."
3 >>> regex = re.compile("ente")
4 >>> regex.search(frase)
5 <re.Match object; span=(13, 17), match='ente'>
```

Sigue en el notebook...

Expresiones regulares: conjuntos de caracteres

Para especificar conjuntos de caracteres dentro de una expresión regular, se utilizan los corchetes y dentro de ellos se ponen todas las opciones de caracteres, [`<caracteres>`], donde `caracteres` son todos los caracteres del conjunto, uno al lado del otro.

Expresiones regulares: conjuntos de caracteres

Para especificar conjuntos de caracteres dentro de una expresión regular, se utilizan los corchetes y dentro de ellos se ponen todas las opciones de caracteres, [`<caracteres>`], donde `caracteres` son todos los caracteres del conjunto, uno al lado del otro.

Si se quiere expresar que un carácter **no** debe pertenecer a un conjunto, se utiliza "[`^<caracteres>`]".

Expresiones regulares: conjuntos de caracteres

Para especificar conjuntos de caracteres dentro de una expresión regular, se utilizan los corchetes y dentro de ellos se ponen todas las opciones de caracteres, `[<caracteres>]`, donde `caracteres` son todos los caracteres del conjunto, uno al lado del otro.

Si se quiere expresar que un carácter **no** debe pertenecer a un conjunto, se utiliza `"[<caracteres>]"`.

Por ejemplo:

- la expresión regular `"[Hhbc]ola"` matchea con `Hola`, `hola`, `bola`, `cola` y `sola`.

Expresiones regulares: conjuntos de caracteres

Para especificar conjuntos de caracteres dentro de una expresión regular, se utilizan los corchetes y dentro de ellos se ponen todas las opciones de caracteres, `[<caracteres>]`, donde `caracteres` son todos los caracteres del conjunto, uno al lado del otro.

Si se quiere expresar que un carácter **no** debe pertenecer a un conjunto, se utiliza `"[^<caracteres>"]`.

Por ejemplo:

- la expresión regular `"[Hhbc]ola"` matchea con `Hola`, `hola`, `bola`, `cola` y `sola`.
- la expresión regular `"[^bc]ola"` matchea con `hola` y `sola`, pero no con `bola` y `cola`.

Expresiones regulares: conjuntos de caracteres

Algunos conjuntos de caracteres se pueden expresar de manera abreviada dentro de los corchetes, por ejemplo:

[a-z]: una letra minúscula

[A-Z]: una letra mayúscula

[a-zA-Z]: una letra

[0-9]: dígito

Expresiones regulares: conjuntos de caracteres

Algunos conjuntos de caracteres se pueden expresar de manera abreviada dentro de los corchetes, por ejemplo:

`[a-z]`: una letra minúscula

`[A-Z]`: una letra mayúscula

`[a-zA-Z]`: una letra

`[0-9]`: dígito

También, existen conjuntos de caracteres predefinidos que se pueden expresar sin necesidad de corchetes, por ejemplo:

`\s`: un carácter de espacio, equivalente a `[\t\n\r\f\v]`

`\d`: un dígito

`\w`: un carácter de palabra

`\W`: la negación de `\w`

Expresiones regulares: caracteres especiales

Hay un conjunto de caracteres que tienen un significado especial dentro de una expresión regular (es decir, no se representan a sí mismos).

Éstos son:

. * ? + ^ \$ \ | () [] { }

Es conveniente memorizarlos. En lo siguiente vamos a ver el significado de cada uno.

Expresiones regulares: caracteres especiales

Hay un conjunto de caracteres que tienen un significado especial dentro de una expresión regular (es decir, no se representan a sí mismos).

Éstos son:

. * ? + ^ \$ \ | () [] { }

Es conveniente memorizarlos. En lo siguiente vamos a ver el significado de cada uno.

¿Y si realmente queremos usar el significado literal de estos caracteres?
Escribimos "\", antes del carácter especial. Esto se llama *escaping*.

Expresiones regulares: caracteres especiales

- Comienzo y fin de un string: $\wedge$ y $\$$, respectivamente.
- El carácter comodín: $.$
- Cuantificadores: $*$, $+$, $?$.
 - $*$ $\rightarrow$ 0 ó más ocurrencias.
 - $+$ $\rightarrow$ 1 ó más ocurrencias.
 - $?$ $\rightarrow$ 0 ó 1 ocurrencia.

Expresiones regulares: caracteres especiales

- Comienzo y fin de un string: $\wedge$ y $\$$, respectivamente.
- El carácter comodín: $.$
- Cuantificadores: $*$, $+$, $?$.
 - $*$ $\rightarrow$ 0 ó más ocurrencias.
 - $+$ $\rightarrow$ 1 ó más ocurrencias.
 - $?$ $\rightarrow$ 0 ó 1 ocurrencia.

Por ejemplo, la expresión regular `".*"` matchea con cualquier string.

Expresiones regulares: caracteres especiales

- El `|` separando dos expresiones regulares representa un **or** lógico.

Métodos de expresiones regulares

Ahora que vimos cómo definir expresiones regulares, veamos los métodos asociados. En lo sucesivo,

- `patron` es un string que representa una expresión regular,
- `regex` es un objeto de la clase `re.Pattern` y
- `texto` es un string donde buscamos el patrón `patron`.

Métodos de expresiones regulares

- `re.compile(patron)`: me genera y devuelve el objeto de tipo `re.Pattern` asociado a `patron`.
- `re.findall(patron, texto)` o `regex.findall(texto)`: devuelve una lista con todas las ocurrencia de `regex` en `texto`.
- `re.match(patron, texto)` o `regex.match(texto)`: devuelve un objeto de tipo `re.Match` si `patron/regex` *matchea* con `texto`, o `None` en caso contrario².
- `re.finditer(patron, texto)` o `regex.finditer(texto)`: devuelve un iterador (i.e. un objeto sobre el cual podemos iterar por ejemplo con un `for`), cada uno de cuyos elementos es un objeto `re.Match` correspondiente a cada una de las ocurrencias de `patron/regex`.

²Notar que el texto **completo** debe matchear con `patron/regex`, no es suficiente que el patrón esté simplemente contenido en `texto`

Métodos de expresiones regulares

- `re.split(patron, texto)` o `regex.split(texto)`: devuelve una lista de `string`'s que resultan de particionar `texto` por todas las ocurrencias de `patron/regex`.
- `re.sub(repl, patron, texto)` o `regex.sub(repl, texto)`: devuelve el `string` que resulta de sustituir todas las ocurrencias de `patron/regex` por el `string repl`.

Extrayendo *grupos* de un string

Dada una expresión regular compuesta por distintas sub-expresiones regulares encerradas entre paréntesis, se pueden extraer las distintas porciones del string que satisfagan cada una:

```
1 >>> import re
2 >>> regex = "(.*)\.(txt|csv|xls|doc)"
3 >>> regex = re.compile(regex)
4 >>> prueba = "archivo_imaginario.xls"
5 >>> nombre = regex.match(prueba).group(1)
6 >>> nombre
7 "archivo_imaginario"
8 >>> extension = regex.match(prueba).group(2)
9 >>> extension
10 "xls"
```

Pregunta 3 (cuestionario)

- a. Escribir una expresión regular que *matchee* con un string que *contiene* una letra **a** seguida de cualesquiera caracteres, y termina en una letra **b**.

- b. Escriba una expresión regular que *matchee* con un string que *contiene* una o más letras **a** seguidas inmediatamente de cero o más dígitos.

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares
- 4 Widgets en Jupyter

Widgets

Los widgets (o *artilugios*) son objetos interactivos que permiten especificar determinados valores utilizando una interfaz gráfica amigable.

Es posible utilizar widgets en Jupyter, a través del módulo `ipywidgets`, el cual se puede instalar desde la consola con `pip` o `conda`:

```
conda install -c anaconda ipywidgets
```

Para utilizar los widgets, se debe importar el módulo `iwidgets` de esta manera:

```
1 import ipywidgets as widgets
```

Notar que el código de esta sección se debe ejecutar en Jupyter notebooks, no en la consola.

Widgets

```
1 import ipywidgets as widgets
2 w = widgets.IntSlider()
3 display(w)
```



Para obtener el valor asociado al widget, se puede hacer

```
1 w.value
2 4
```

Podemos dotar a nuestros widgets de reactividad mediante la función `interact`:

```
1 import ipywidgets as widgets
2 from ipywidgets import interact
3
4 def f(x):
5
6     #####
7     # Lo que queremos hacer
8     #####
9
10 interact(f, x=widgets.IntSlider(min=0, max=10))
```

¡Fin de la clase 4!